

Exemple de creació d'un XSD

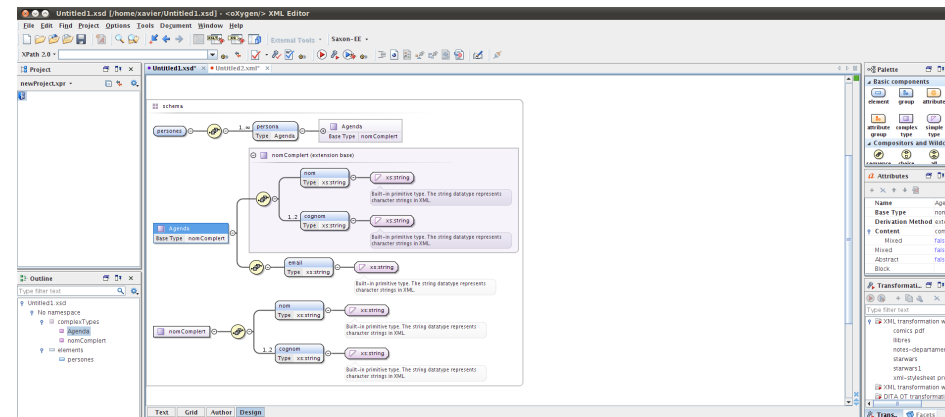
Es poden crear definicions de vocabularis XSD a partir de la idea del que volem que continguin les dades o bé a partir d'un fitxer XML de mostra.

Enunciat: CAS 1

En un centre en què només s'imparteixen els cicles formatius d'SMX i ASIX, quan han d'entrar les notes als alumnes ho fan creant un arxiu XML com el següent:

```
1 <?xml version="1.0" ?>
2 <classe modul="3" nommodul="Programació bàsica">
3   <curs numero="1" especialitat="ASIX">
4     <professor>
5       <nom>Marcel</nom>
6       <cognom>Puig</cognom>
7     </professor>
8     <alumnes>
9       <alumne>
10        <nom>Filomeno</nom>
11        <cognom>Garcia</cognom>
12        <nota>5</nota>
13      </alumne>
14      <alumne delegat="si">
15        <nom>Frederic</nom>
16        <cognom>Pi</cognom>
17        <nota>3</nota>
18      </alumne>
19      <alumne>
20        <nom>Manel</nom>
21        <cognom>Puigdevall</cognom>
22        <nota>8</nota>
23      </alumne>
24    </alumnes>
25  </curs>
26 </classe>
```

A la secretaria necessiten que es generi la definició del vocabulari en XSD per comprovar que els fitxers que reben són correctes.



1.Anàlisi

La primera fase normalment consisteix a analitzar el fitxer per determinar si hi ha parts que poden ser susceptibles de formar un tipus de dades. En l'exercici es pot veure que tant per al professor com per als alumnes les dades que contenen són semblants (els alumnes afegeixen la nota), i per tant es pot crear un tipus de dades que farà més simple el disseny final.

Per tant, en l'arrel **podem crear el tipus complex persona**, que contindrà el nom i el cognom.

```
1 <xs:complexType name="persona">
2   <xs:sequence>
3     <xs:element name="nom" type="xs:string"/>
4     <xs:element name="cognom" type="xs:string"/>
5   </xs:sequence>
6 </xs:complexType>
```

Com que els alumnes són iguals però amb un element extra **es pot aprofitar el tipus persona estenent-lo**.

```
1 <xs:complexType name="estudiant">
2   <xs:complexContent>
3     <xs:extension base="persona">
4       <xs:sequence>
5         <xs:element name="nota">
6           ...
7         </xs:element>
8       </xs:sequence>
9     </xs:extension>
10   </xs:complexContent>
11 </xs:complexType>
```

Les notes no poden tenir tots els valors (només d'1 a 10), o sigui, que es pot afegir una restricció al tipus enter.

```
1 <xs:element name="nota">
2   <xs:simpleType>
3     <xs:restriction base="xs:integer">
4       <xs:maxInclusive value="10"/>
5       <xs:minInclusive value="1"/>
6     </xs:restriction>
7   </xs:simpleType>
8 </xs:element>
```

2.Desenvolupament

- L'arrel sempre serà de tipus complex perquè conté atributs i tres elements, de manera que es pot començar la declaració com una seqüència d'elements.

```
1 <xs:element name="classe">
2   <xs:complexType>
3     <xs:sequence>
4       <xs:element name="curs">
5         ...
6       </xs:element>
7       <xs:element name="professor" type="persona"/>
8       <xs:element name="alumnes">
9         ...
10      </xs:element>
11    </xs:sequence>
12  </xs:complexType>
13 </xs:element>
```

- S'han deixat els elements <curs> i <alumnes> per desenvolupar-los, mentre que l'element <professor> ja es pot tancar perquè s'ha definit el tipus persona.
- L'element <curs> no té contingut i, per tant, serà de tipus complex. A més, s'hi han de definir dos atributs.

```
1 <xs:element name="curs">
2   <xs:complexType>
3     <xs:attribute name="numero" use="required">
4       ...
5     </xs:attribute>
6     <xs:attribute name="especialitat" use="required">
7       ...
8     </xs:attribute>
9   </xs:complexType>
10 </xs:element>
```

Els atributs s'han de desenvolupar perquè no poden tenir tots els valors:

- numero només pot ser “1” o “2”.
- especialitat serà “SMX” o “ASIX”.

La manera més adequada per fer-ho serà **restringir els valors amb una enumeració**.

```
1 <xs:attribute name="numero" use="required">
2   <xs:simpleType>
3     <xs:restriction base="xs:integer">
4       <xs:enumeration value="1"/>
5       <xs:enumeration value="2"/>
6     </xs:restriction>
7   </xs:simpleType>
8 </xs:attribute>
9 <xs:attribute name="especialitat" use="required">
10   <xs:simpleType>
11     <xs:restriction base="xs:string">
12       <xs:enumeration value="ASIX"/>
13       <xs:enumeration value="SMX"/>
14     </xs:restriction>
15   </xs:simpleType>
16 </xs:attribute>
```

Ja només queda per desenvolupar <alumnes>, que té una repetició d'elements <alumne>, del qual ja n'hi ha un tipus.

```
1 <xs:element name="alumne">
2   <xs:complexType>
3     <xs:sequence>
4       <xs:element name="alumne" type="estudiant"
5         maxOccurs="unbounded" minOccurs="1"/>
6     </xs:sequence>
7   </xs:complexType>
8 </xs:element>
```

El resultat final:

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
3
4  <xs:complexType name="persona">
5    <xs:sequence>
6      <xs:element name="nom" type="xs:string"/>
7      <xs:element name="cognom" type="xs:string"/>
8    </xs:sequence>
9  </xs:complexType>
10
11 <xs:complexType name="estudiant">
12   <xs:complexContent>
13     <xs:extension base="persona">
14       <xs:sequence>
15         <xs:element name="nota">
16           <xs:simpleType>
17             <xs:restriction base="xs:integer">
18               <xs:maxInclusive value="10"/>
19               <xs:minInclusive value="1"/>
20             </xs:restriction>
21           </xs:simpleType>
22         </xs:element>
23       </xs:sequence>
24       <xs:attribute name="delegat" use="optional">
25         <xs:simpleType>
26           <xs:restriction base="xs:string">
27             <xs:enumeration value="si"/>
28           </xs:restriction>
29         </xs:simpleType>
30       </xs:attribute>
31     </xs:extension>
32   </xs:complexContent>
33 </xs:complexType>
34
35 <xs:element name="classe">
36   <xs:complexType>
37     <xs:sequence>
38       <xs:element name="curs">
39         <xs:complexType>
40           <xs:sequence>
41             <xs:element name="professor" type="persona"/>
42             <xs:element name="alumnes">
43               <xs:complexType>
44                 <xs:sequence>
45                   <xs:element name="alumne" type="estudiant"
46                     maxOccurs="unbounded" />
47                 </xs:sequence>
48               </xs:complexType>
49             </xs:element>
50           </xs:sequence>
51           <xs:attribute name="numero" use="required">
52             <xs:simpleType>
53               <xs:restriction base="xs:integer">
54                 <xs:enumeration value="1"/>
55                 <xs:enumeration value="2"/>
56               </xs:restriction>
57             </xs:simpleType>
58           </xs:attribute>
59           <xs:attribute name="especialitat" use="required">
60             <xs:simpleType>
61               <xs:restriction base="xs:string">
62                 <xs:enumeration value="ASIX"/>
63                 <xs:enumeration value="SMX"/>
64               </xs:restriction>
65             </xs:simpleType>
66           </xs:attribute>
67         </xs:complexType>
68       </xs:element>
69     </xs:sequence>
70     <xs:attribute name="modul" use="required" type="xs:integer" />
71     <xs:attribute name="nommodul" use="required" type="xs:string" />
72   </xs:complexType>
73 </xs:element>
74 </xs:schema>
```

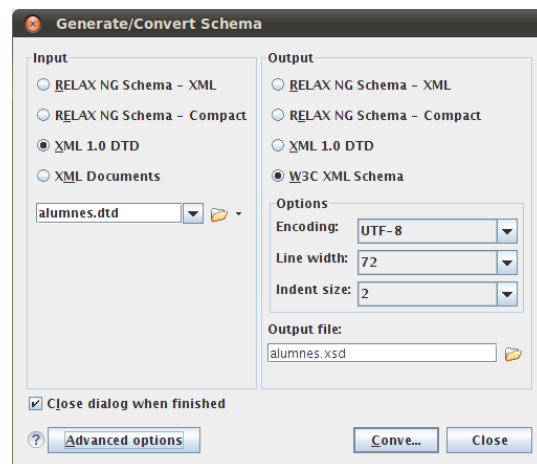
Conversió entre esquemes

Una de les eines que s'han de tenir en compte a l'hora de fer esquemes és que ja hi ha eines que permeten convertir els llenguatges de definició d'esquemes d'un format a un altre.

A pesar d'això **sempre s'hauran de revisar els resultats per comprovar** que no s'ha perdut cap significat important en fer la conversió, ja que els sistemes automàtics no són perfectes.

La majoria dels **editors XML porten alguna eina per fer la conversió** però també hi ha eines específiques per fer conversions, com el programa de **codi obert Trang**.

L'oXygen



Trang

Programa de codi obert que permet convertir les definicions de vocabularis fetes en un sistema a un altre. Pot convertir els llenguatges següents:

- RELAX NG (XML syntax)
- RELAX NG compact syntax
- XML 1.0 DTD
- W3C XML Schema

****ACTIVITAT****

Per fer la conversió simplement **hem d'especificar el fitxer que es vol convertir i en què es vol convertir**. L'única condició és que l'esquema d'origen no sigui un XML Schema.

Es pot convertir fent servir fitxers que tinguin les extensions per defecte: DTD (.dtd), XML Schemas (.xsd), RELAX NG (.rng), RELAX NG compact (.rnc).

Amb això es crearà **alumnes.xsd** a partir d'alumnes.dtd.

```
$ trang alumnes.dtd alumnes.xsd
```

O bé especificant explícitament quin és el tipus de fitxer d'entrada (-I) i quin el de sortida (-O).

```
$ trang -I dtd -O xsd alumnes.dtd alumnes.xsd
```

Una de les característiques interessants que té és que pot crear el vocabulari basant-se en fitxers XML de mostra. A partir del fitxer XML, alumnes.xml, es pot fer que el Trang generi automàticament l'XML Schema que el validaria executant:

```
$ trang alumnes.xml alumnes.xsd
```

Si es tenen diversos documents XML també pot generar l'esquema associat amb aquests:

```
$ trang *.xml alumnes.xsd
```